恒生电子股份有限公司

C、C++编码规范

2021 年 12 月

说　明

本文档中所包含的信息属于商业机密信息，如无恒生电子股份有限公司的书面许可，任何人都无权复制或利用。

密级：内部

目 录

一、概述.....3
二、约束等级定义.....3
三、布局结构规范.....3
1 一般规则.....3
2 文件头部说明.....4
3 函数头部说明.....4
4 头文件结构.....5
5 源文件结构.....6
6 MAKEFILE 文件结构.....7
7 目录结构.....7
四、书写排版规范.....8
1 一般规则.....8
2 空格.....8
3 空行.....10
4 对齐.....12
5 缩行.....15
6 长行拆分.....17
五、命名规范.....18
1 一般规则.....18
2 标示符命名.....18
3 文件命名.....21
六、语言规范.....22
1 常量.....22
2 变量.....22
3 预处理.....22
4 宏.....23
5 表达式.....24
6 语句.....30
7 注释.....40
8 类.....43
9 类型转换.....49
10 指针和数组.....54
11 异常.....57
七、其它规范.....59
八、64 位程序移植规范.....74
九、建议反馈途径.....75
十、修订记录.....75

一、 概述

本规范旨在为全公司 C/C++程序的开发者提供规范化的指导，可用于程序员个人环境以及研发团队集成环境等场合的代码规范化检查。

二、 约束等级定义

约束等级	约束效力	强制性
【强制】	必须做的项，改进有意义，不改可能造成程序问题或者理解错误；或者违反该项将被认为代码存在严重缺陷	全公司所有 C/C++编码团队必须遵守
【推荐】	改进有意义，提升产品质量，并增强可读性，违反该项即被认为代码存在轻微缺陷	各 C/C++编码团队根据具体产品特性的不同， 选择性地遵守
【参考】	代码更整齐，或者增强可读取，违反该项可被认为代码存在优化空间	各 C/C++编码团队从产品持续优化及人员技 能提升的角度，参考使用

三、 布局结构规范

1 一般规则

【强制】每个源程序文件的头部必须包含文件头部说明和修改记录（与头部说明合并）

检查方式：人工检查

原因：提升代码可读性

【强制】每个函数头部必须包含函数头部说明，内部函数头部至少需要描述函数的主要功能

检查方式：人工检查

原因：提升代码可读性

【强制】每个源程序文件的尾部增加一行空行

检查方式：工具

原因：提升代码可读性

2　文件头部说明

【强制】源程序的文件头部说明必须包含系统名称、模块名称、文件名称、软件版权、功能说明、系统版本、开发人员、开发时间和修改记录等几个部分，审核人员、审核日期和相关文档为可选项

检查方式：人工检查

原因：提升代码可读性

【强制】源程序的修改记录说明必须包含修改日期、修改人员、修改说明，每次修改为一条记录

检查方式：人工检查

原因：提升代码可读性

3　函数头部说明

【推荐】函数的头部说明必须包含函数名称、函数功能、输入参数、输出参数、返回值等几个部分，对环境引用、对环境影响、内部变量说明、内部变量说明和被调函数列表为可选项，说明格式建议参照 doxygen 等工具的规范，比如@param 标记参数的意义，@return 描述返回意义。各部分次序如下：

函数名称	本函数的名称
函数功能	简述函数的功能
输入参数	列出主要的、不易理解的输入参数的含义和取值范围
输出参数	列出主要的、不易理解的输出参数的含义和取值范围
返回值	函数返回值的含义和取值范围
调用说明（可选的）	是否线程安全
对环境引用（可选的）	需要引用的系统环境，如全局变量、环境变量等
对环境影响（可选的）	可能会改变的系统环境，如全局变量、环境变量等
内部变量说明(可选的)	本函数内部的重要的变量说明
内部变量说明(可选的)	本函数内部主要调用的函数清单
被调函数列表(可选的)	调用本函数的主要函数清单

检查方式：人工检查

原因：提升代码可读性

4 头文件结构

【推荐】头文件各部分的书写顺序如下：

文件头部说明	见前
#define 保护	
包含的头文件	#include
常量定义	const
宏定义	#define
类型定义	typedef
结构、联合定义	struct 或 union
外部变量声明	extern
全局变量定义	
函数原型声明	

检查方式：人工检查

原因：提升代码可读性

【强制】所有头文件都应该使用#define 来防止头文件被多重包含，命名格式为：_<FILE>_H_

检查方式：工具

原因：防止头文件被多重包含

【强制】#include 之前不得声明符号

检查方式：工具

原因：容易被忽略，不利于代码维护

反例：

```
int32_t a;
#include <stdio.h>
int32_t main()
{
    …
}
```

【推荐】对外头文件名称应包含产品简称

检查方式：人工检查

原因：避免不同产品的头文件冲突，实际使用时出现引用错误

正例：ldp_host.h

反例：host.h

5 源文件结构

【推荐】源文件各部分的书写顺序如下：

文件头部说明	见前
包含的头文件	#include
常量定义	const
宏定义	#define
类型定义	typedef
结构、联合定义	struct 或 union
外部变量声明	extern
全局变量定义	
函数原型声明	
一般函数	
main 函数	

检查方式：人工检查

原因：提升代码可读性

【推荐】头文件引用按照项目目录结构，避免使用..(上级目录)

检查方式：人工检查

原因：提升代码可读性，同时目录变动时，调整最少

正例：

源码目录结构为：

```
Sources
|-----plugins
    |------A
        |------a.h
    |------B
        |------b.cpp
```

b.cpp 中需要引用 a.h 推荐的引用方式为： #include <plugins/A/a.h>

【推荐】头文件包含顺序建议如下：

相关头文件（一般为 cpp 文件同名的.h 文件）

C 系统文件

C++ 系统文件

其他库的 .h 文件

本项目内 .h 文件

检查方式：人工检查

原因：提升代码可读性

【建议】头文件引用尽量在源码文件中引用，避免头文件相互依赖

检查方式：人工检查

原因：避免头文件相互依赖

6 Makefile 文件结构

【推荐】编译生成的中间文件（.o 或者.obj 文件）最好放到独立的目录下。比如同一份源码根据编译选项生成不同的目标程序时，编译不同目标程序生成的中间文件分开存储

检查方式：人工检查

原因：避免同时编译时，链接错误（makefile 编译时未清理中间文件）

7 目录结构

【强制】在源程序目录下，无用的文件必须移走或删除。

检查方式：人工检查

原因：不利于后期代码质保维护；甚至可能会被编译链接到目标程序中，带来意想不到的后果

【推荐】保持目录结构清晰，头文件与源文件要存放在不同的子目录下，系统公用的头文件与某程序私有的头文件也要分别放在不同的子目录中。

检查方式：人工检查

原因：目录结构清晰，方便阅读理解整个项目代码

正例：

```
Sources
|-----bin/          执行文件目录，包含配置信息，可运行程序等
|-----include/      系统公用头文件，模块接口定义文件
|-----utilities/        系统公用类的实现，包含头文件和实现文件
|-----plugins/          插件（模块）源码
|-----3rd/              依赖的第三方开源代码或组件
|-----demo/             开发示例
```

四、 书写排版规范

1 一般规则

【强制】不允许把多个短语句写在一行中，即一行最多只写一条语句

检查方式：人工检查

原因：提升代码可读性

【建议】合理使用空格、空行、对齐、缩行、长行拆分等方式格式化源代码，使程序舒展，可读

检查方式：人工检查

原因：提升代码可读性

2 空格

【强制】关键字之后要留空格。象 const、define、case 等关键字之后至少要留一个空格，否则无法辨析关键字。象 if、for、while、switch 等关键字之后应留一个空格再跟左括号‘(’，以突出关键字

检查方式：人工检查

原因：提升代码可读性

正例：

```
if (year >= 2000)
```

反例：

```
if(year>=2000)
```

【强制】‘,’之后要至少要留一个空格

检查方式：人工检查

原因：提升代码可读性

【强制】如果‘;’不是一行的结束符号，其后要留一个空格

检查方式：人工检查

原因：提升代码可读性

正例：

```
for (i = 0; i < 10; i++)
```

反例：

```
for(i=0;i<10;i++)
```

【强制】赋值操作符、比较操作符、算术操作符、逻辑操作符、位域操作符，如“=”、“+=” “>=”、“<=”、“+”、“*”、“%”、“&&”、“||”、“<<”，“^”等二元操作符的前后应当各加一个空格

检查方式：人工检查

原因：提升代码可读性

正例：

```
if ((a >= b) && (c <= d))
```

反例：

```
if(a>=b&&c<=d)
```

【建议】注释前后适当加一个或若干个空格

检查方式：人工检查

原因：提升代码可读性

正例：

```
/* 我的注释 */
```

反例：

```
/*我的注释*/
```

【强制】函数名之后紧跟左括号‘(’，中间不要留空格，以与关键字区别

检查方式：人工检查

原因：提升代码可读性

正例：

```
void Func1(int x, int y, int z);
```

反例：

```
void Func1 (int x,int y,int z);
```

【强制】‘(’之后和‘)’、‘,’、‘;’之前都不留空格

检查方式：人工检查

原因：提升代码可读性

【强制】一元操作符如“!”、“~”、“++”、“--”、“*” （指针取值符）、“&”（地址运算符）等前后不得加空格

检查方式：人工检查

原因：提升代码可读性

正例：

```
int *x = &y;
```

反例：

```
int * x = & y;
```

【强制】“[ ]”、“.”、“->”这类操作符前后不加空格

检查方式：人工检查

原因：提升代码可读性

正例：

```
array[5] = 0;
```

反例：

```
array [ 5 ] = 0;
```

3 空行

【强制】每个函数定义结束之后都要加一个或若干个空行

检查方式：人工检查

原因：提升代码可读性

正例：

```
空行
void Function1(…)
{
  …
}
空行
void Function2(…)
{
  …
}
空行
void Function3(…)
{
  …
}
空行
```

【强制】在一个函数体内，变量定义与函数语句之间要加空行

检查方式：人工检查

原因：提升代码可读性

正例：

```
void Function1(…)
{
    int n;
    空行
   statement1;
   …
}
```

【推荐】逻辑上密切相关的语句之间不加空行，其它地方应加空行分隔

检查方式：人工检查

原因：提升代码可读性

正例：

```
空行
while (condition)
{
    statement1;
    空行
    if (condition)
    {
        statement2;
    }
    else
    {
        statement3;
    }
     空行
    statement4;
}
```

4　对齐

【强制】程序的分界符‘{’和‘}’永远都单独成行并且位于同一列，同时与引用它们的语句左对齐

检查方式：人工检查

原因：提升代码可读性

正例：

```
void Function(int x)
{
  …  program code
}

if (condition)
{
  …  program code
```

```
}
else
{
  …  program code
}

for (initialization; condition; update)
{
  …  program code
}

while (condition)
{
  …  program code
}

do
{
  …  program code
}
while (condition);
```

反例：

```
void Function(int x){
…  program code
}

if (condition){
…  program code
}
else{
…  program code
}
```

```
for (initialization;condition; update){
…  program code
}

while (condition){
…  program code
}

do{
…  program code
}while (condition);
```

【强制】CASE 语句语的 case 部分与 switch 的‘{’列对齐

检查方式：人工检查

原因：提升代码可读性

正例：

```
switch (variable)
{
case value1 :
    …
    break;
case value2 :
    …
    break;
…
default :
    …
    break;
}
```

反例：

```
switch (variable)
```

```
{
    case value1 : …
        break;
    case value2 : …
        break;
    …
    default :  …
      break;
}
```

5　缩行

【强制】用缩行显示程序结构，使排版整齐，缩进量统一使用 4 个字符

检查方式：人工检查

原因：提升代码可读性，保证各个平台看到的代码格式都是统一的

【推荐】最好用 4 个空格而不是 Tab 来实现缩行

检查方式：人工检查

原因：提升代码可读性，保证各个平台看到的代码格式都是统一的

【强制】在函数体的开始、结构/联合的定义、枚举的定义以及循环、判断等语句中的代码都要采用缩行。

检查方式：人工检查

原因：提升代码可读性

正例：

```
void Function(int x)
{
    …  program code
}
struct tagMyStruct
{
    int a;
    int b;
```

```
    int c;
};
if (condition)
{
…  program code
}
else
{
…  program code
}
```

反例：

```
void Function(int x)
{
…  program code
}
struct tagMyStruct
{
int a;
int b;
int c;
};
if (condition)
{
…  program code
}
else
{
…  program code
}
```

【强制】同层次的代码在同层次的缩进层上

检查方式：人工检查

原因：提升代码可读性

6　长行拆分

【强制】（代码行不宜过长），较长的语句、表达式等要分成多行书写

检查方式：人工检查

原因：提升代码可读性

【推荐】长表达式要在低优先级操作符处划分新行，操作符放在新行之首（以便突出操作符）。拆分出的新行要进行适当的缩进，使排版整齐，语句可读。

检查方式：人工检查

原因：提升代码可读性

正例：

```
if ((very_longer_variable1 >= very_longer_variable12)
   && (very_longer_variable3 <= very_longer_variable14)
   && (very_longer_variable5 <= very_longer_variable16))
{
    dosomething();
}

for (very_longer_initialization;
     very_longer_condition;
     very_longer_update)
{
    dosomething();
}
```

【推荐】如果函数中的参数较长，则要进行适当的划分

检查方式：人工检查

原因：提升代码可读性

正例：

```
void Function(float very_longer_var1,
              float very_longer_var2,
```

```
                    float very_longer_var3);
```

五、 命名规范

1 一般规则

【建议】命名应当直观且可以拼读，可望文知意，便于记忆和阅读。标识符最好采用英文单词或其组合，避免使用拼音。程序中的英文单词一般不会太复杂，用词应当准确

检查方式：人工检查

原因：提升代码可读性

【强制】变量命名的时候，单词首字母都大写，然后在最前面增加变量类型的前缀

检查方式：人工检查

原因：提升代码可读性

正例：

```
int iMaxValue;
```

【推荐】命名的长度应当符合“min-length && max-information”原则，标识符的长度一般不要过长，较长的单词可通过去掉“元音”形成缩写

检查方式：人工检查

原因：提升代码可读性

【推荐】尽量避免名字中出现数字编号，如 iValue1,iValue2 等，除非逻辑上的确需要编号。这是为了防止程序员偷懒，不肯为命名动脑筋而导致产生无意义的名字（因为用数字编号最省事）

检查方式：人工检查

原因：提升代码可读性

2 标示符命名

【强制】程序中不得出现仅靠大小写区分的相似的标识符

检查方式：人工检查

原因：提升代码可读性

反例：

```
int  x,  X;          // 变量 x 与 X 容易混淆
void foo(int x);   // 函数 foo 与 FOO 容易混淆
void FOO(float x);
```

【强制】局部变量不得与全局变量重名，尽管两者的作用域不同而不会发生语法错误，但会使人误解

检查方式：工具检查

原因：提升代码可读性，避免出错

【推荐】变量的名字应当使用“名词”或者“形容词＋名词”。

检查方式：人工检查

原因：提升代码可读性

正例：

```
float  fValue;
float  fOldValue;
float  fNewValue;
```

【推荐】全局函数的名字应当使用“动词”或者“动词＋名词”（动宾词组）。各单词连写，词首字母大写

检查方式：人工检查

原因：提升代码可读性

正例：

```
void DrawBox();
```

【推荐】用正确的反义词组命名具有互斥意义的变量或相反动作的函数等

检查方式：人工检查

原因：提升代码可读性

正例：

```
int iMinValue;
int iMaxValue;
```

```
int SetValue(…);
int GetValue(…);
```

【强制】所有宏定义、枚举常数、常量全用大写字母命名，并加上项目或者模块简称前缀，用下划线分割单词

检查方式：工具

原因：提升代码可读性，同时可以避免冲突

正例：

```
const int LDP_MAX_LENGTH = 100;
#define LDP_FILE_PATH “/usr/tmp”
```

【强制】静态变量加前缀 s_（表示 static）

检查方式：人工检查

原因：提升代码可读性

正例：

```
void Init(…)
{
    static int s_iInitValue;
    …
}
```

【强制】全局变量加前缀 g_（表示 global）

检查方式：工具

原因：提升代码可读性

正例：

```
int g_iCount;
```

【强制】C++的类成员变量命名，增加”m_”前缀，便于区分函数中类成员变量和局部变量

检查方式：人工检查

原因：提升代码可读性

正例：

```
class A{
private:
    int m_iValue;
};
```

【强制】接口命名加“I”前缀

检查方式：人工检查

原因：提升代码可读性

正例：

```
class IMath{
public:
    virtual int Add(int iLeftValue, int iRightValue) = 0;
};
```

3　文件命名

【强制】头文件名或#include 指令中不得出现’，”，/ *，//，\字符

检查方式：工具

原因：C/C++语言不支持

反例：

```
#include <header’file.h>
#include <”headerfile.h”>
#include <directory\\headerfile.h>
```

【推荐】项目组应采用相同的风格命名源文件，全小写风格：组成文件名的单词全采用小写字母，单词间用下划线分隔，避免取一些比较通俗的文件名；首字母大写风格：组成文件名的单词的首字母大写

检查方式：人工检查

原因：风格统一，从文件名可以推断文件内容

正例：

```
rcm_factory.cpp
thread_mutext.cpp
```

反例：

```
main.cpp
public.cpp
```

六、 语言规范

1 常量

【推荐】需要对外公开的常量放在头文件中，不需要对外公开的常量放在定义文件的头部。为便于管理，可以把不同模块的常量集中存放在一个公共的头文件中

检查方式：人工检查

原因：方便管理

【推荐】如果某一常量与其它常量密切相关，应在定义中包含这种关系，而不应给出一些孤立的值

检查方式：人工检查

原因：提升代码可读性

正例：

```
const  float   RADIUS = 100.0;
const  float   DIAMETER = RADIUS * 2;
```

2 变量

【强制】声明变量的同时对变量进行初始化，严禁使用未经初始化的变量

检查方式：工具

原因：避免因为访问未经初始化的变量，导致程序行为不确定

3 预处理

【参考】#if 或#elif 预处理指令的控制表达式应为 0 或 1

检查方式：工具

原因：提升代码可读性

反例：

```
#if 10 //不为 0 或者 1
#endif
```

【推荐】#if 或#elif 的控制表达式中使用的所有标识符使用前应先定义

检查方式：工具

原因：未定义的标识符不能期望值为 0

正例：

```
#if defined M
#if M == 0

#endif
#endif
```

反例：

```
#if M == 0     //M 没有定义
#endif
```

4　宏

【强制】宏体和参数都要加上括号

检查方式：工具

原因：不使用括号会导致有些场景无法得到期望值

正例：

```
#define MAX(a, b)  ((a) > (b) ? (a) : (b))
```

反例：

```
#define MAX(a, b)  a > b ? a : b
```

【推荐】为杜绝可能带来的副作用，尽量避免采用函数宏

检查方式：工具

原因：存在副作用

【推荐】C ++标准库中保留的标识符，宏和函数不得被定义或取消定义

检查方式：工具

原因：会导致其它使用该宏的地方出现问题

反例：

```
#undef __TIME__
#define __LINE__ 20
```

【强制】宏参数中不能有预处理指令

检查方式：工具

原因：会把整个预处理指令相关代码传下去

反例：

```
#define M(A) printf (#A)
#include <stdio.h>
void main (void)
{
  M(
    #ifdef SW
    "Message 1"
    #else
    "Message 2"
    #endif
  );
}
```

【强制】紧随＃运算符之后的宏参数不应立即跟随##运算符

检查方式：工具

原因：可能会无法得到期望值

正例：

```
#define A(x) #x
#define B(x, y) x ## y
```

反例：

```
#define C(x, y) #x ## y
```

【推荐】用 const 定义常量，而不要用宏

检查方式：工具

原因：编译器可以进行类型安全检查

5　表达式

【推荐】如果代码行中的运算符比较多，用括号确定表达式的操作顺序，避免使

用默认的优先级

检查方式：工具

原因：提升代码可读性

正例：

```
leap_year = (((year % 4) == 0) && ((year % 100) != 0)) || ((year % 400) == 0);
```

【强制】不要编写太复杂的复合表达式

检查方式：人工检查

原因：提升代码可读性

反例：

```
i = a >= b && c < d && c + f <= g + h;
```

【强制】不要有多用途的复合表达式

检查方式：人工检查

原因：提升代码可读性

反例：

```
d = (a = b + c) + r;
```

该表达式既求 a 值又求 d 值。应该拆分为两个独立的语句：

```
a = b + c;
d = a + r;
```

【推荐】尽量避免含有否定运算的条件表达式

检查方式：人工检查

原因：提升代码可读性

正例：

```
if (num < 10)
```

反例：

```
if (!(num >= 10))
```

【推荐】类型为 bool 的表达式不得用作赋值运算符=，逻辑运算符&&、||、!，

等式运算符==、!=，条件运行符以外的内置运算符的操作数

检查方式：工具

原因：把 bool 类型当作其它类型使用没有意义

正例：

```
bool b1 = true;
bool b2 = false;
if (b1 == false)
if (b1 == b2)
if (b1 != b2)
if (b1 && b2)
if (!b1)
int32_t a = b1 ? 3:7;
```

反例:

```
bool b1 = true;
bool b2 = false;
if (b1 & b2)
if (b1 < b2)
if (~b1)
if (b1 ^ b2)
```

【推荐】条件运算符的第一个操作数必须为 bool 类型

检查方式：工具

原因：提升代码可读性

正例：

```
int32_t a;
bool b1;
int32_t b = b1 ? 6:10;
int32_t c = (a < 10)? 6:10;
```

反例：

```
int32_t a;
int32_t b = a ? 6:10;
```

【强制】按位运算符的非常量操作数应具有相同的基础类型

检查方式：工具

原因：避免隐式类型转换

反例：

```
uint8_t mask = ~(0x10);
uint16_t value;
value ^= mask;    // value 和 mask 类型不一致
```

【推荐】按位运算符应只应用于无符号基础类型的操作数

检查方式：工具

原因：有符合的最高位为符号位，不建议位运算

说明：按位运算符包括~,<<,<<=,>>,>>=,&, &=,^,^=,|和|=

反例：

```
uint16_t a;
int16_t b;
if((a & b) == 0x1234U)
if(~b == 0x1234U)
```

【参考】逻辑&&或||运算符的每个操作数应该是后缀表达式

检查方式：工具

原因：提高代码可读性

说明：要求对操作数进行适当的括号括起来，如果表达式只包括||或者&&的运算符，也可以不需要括号

正例：

```
bool a;
if ( (x == 0) && a)
if (x || y || z) //只包含||运算符
if (x || (!y))
```

反例：

```
bool a;
```

```
if (x == 0 && a)
if (x || y && z)
if (x || !y)
```

【推荐】不建议在表达式中使用++和—

检查方式：工具

原因：表达式中使用++和—会代码可读性变低及导致不确定的行为

反例：

```
uint8_t a, b, c;
a = ++b + c--;
```

【参考】将非布尔操作数与！，&&或||一起使用无意义

检查方式：工具

原因：隐式类型转换

反例：

```
int32_t c, d;
if ( 1 && (c < d))
if (!0)
```

【参考】一元负运算符不得应用于无符号类型

检查方式：工具

原因：结果仍为无符号类型

反例：

```
uint8_t a = -1U; // uint8_t 为无符号
int32_t b = -a;
uint32_t c = 1U;
int64_t d = -c;
```

【推荐】移位操作符右侧的值超过了基础类型的范围

检查方式：工具

原因：使用超出范围

反例：

```
uint8_t a ;
a = a << 9;    // a 只有 8 位
```

【参考】有副作用的操作符在逻辑运算符&&和||的右边

检查方式：工具

原因：可能导致不确定的结果

说明：在逻辑运行符&&和||的右边可能被执行到，也可能不会执行到

反例：

```
bool b
int32_t x,j;
if(b && (x == i++))
```

【参考】不建议使用逗号表达式

检查方式：工具

原因：提升代码可读性

【参考】子表达式中不得使用赋值运算符

检查方式：工具

原因：可能是编写代码时候的疏忽导致

反例：

```
int32_t a, b, c;
a = b = c;

if ((a = b) != 0)
{
}
if (x = y)
{
}
```

【参考】关系运算符应返回布尔值

检查方式：工具

原因：关系运算符没有必要返回其它类型

说明：关系运算符包括<,<=,>,>=,==和!=

反例：

```
class A
{
public:
    int32_t operator !=(A const & lhs, A const & rhs);   //返回类型不对
}
```

【强制】操作符和类型不匹配

检查方式：工具

原因：语法错误

反例：

```
class C
{
    int32_t a;
    int32_t b;
};
C c;
c->a = 1;
float f = 10.0;
f >> 2;    //浮点数右移
```

6　语句

【强制】if 语句本身自占一行，执行语句不得紧跟其后。不论执行语句有多少都要加{}

检查方式：人工检查

原因：提升代码可读性，避免出错

正例：

```
if (width < height)
{
```

```
        dosomething();
    }
```

反例：

```
    if (width < height) dosomething();
```

【强制】不可将浮点变量用“==”或“！=”与任何数字比较

检查方式：工具

原因：浮点数比较大小应该添加对应精度

正例：

```
    if ((x >= -FLT_EPSINON) && (x <= FLT_EPSINON))
```

反例：

```
    if (x == 0.0)
```

其中 FLT_EPSINON 是允许的误差（即精度）

【强制】应当将指针变量用“==”或“！=”与 NULL 比较

检查方式：人工检查

原因：提升代码可读性

正例：

```
    if (p == NULL)
    if (p != NULL)
```

反例：

```
    if (p == 0)
    if (p != 0)
    if (p)
    if (!p)
```

【推荐】每个 case 语句的结尾绝对不要忘了加 break，否则将导致多个分支重叠（除非有意使多个分支重叠）

检查方式：工具

原因：避免由于疏忽导致分支重叠

【强制】不要忘记最后那个 default 分支。即使程序真的不需要 default 处理，也应该保留语句

检查方式：工具

原因：提升程序健壮性

【强制】对于没有循环体的循环语句，使用{}来代替空语句

检查方式：工具

原因：提升代码可读性

正例：

```
while （…）
{
}
for （…）
{
}
```

反例：

```
while （…）;
for （…）;
```

【推荐】在多重循环中，如果有可能，应当将最长的循环放在最内层，最短的循环放在最外层，以减少 CPU 跨切循环层的次数

检查方式：人工检查

原因：减少跳转，提升程序运行效率

正例：

```
for (i=0; i<5; i++ )
{
    for (j=0; j<100; j++)
    {
        …
    }
}
```

反例：

```
for (j=0; j<100; j++)
{
    for (i=0; i<5; i++ )
    {
        …
    }
}
```

【推荐】建议 for 语句的循环控制变量的取值采用“半开半闭区间”写法

检查方式：人工检查

原因：符合习惯，提升代码可读性

正例：

```
for (n = 0; n < 10; n++)
{
  …
}
```

反例：

```
for (n = 0; n <= 9; n++)
{
  …
}
```

【推荐】少用、慎用 goto 语句

检查方式：工具

原因：提升代码可读性

【推荐】goto 的标签不在当前语句块中

检查方式：工具

原因：该写法会降低代码可读性

反例：

```
void f1 ()
{
```

```
    int32_t j = 0;
    goto L1;
    for (j = 0; j < 10; ++j)
    {
    L1:
        j;
    }
}
```

【强制】goto 到前面的标签

检查方式：工具

原因：该写法会降低代码可读性

反例：

```
void f (void)
{
    int32_t j = 0;
L1:
    ++j;
    if (10 == j)
    {
    ...
    }
    goto L1;
    ++j;
}
```

【推荐】对于变量在自增(++i 或 i++) 或自减 (--i 或 i--) 后表达式的值又没有没用到的情况下，使用前置的自增（自减），尤其是迭代器和其他模板对象

检查方式：人工检查

原因：避免创建临时对象

正例：

```
auto iter = map1.begin();
```

```
for (;iter != map1.end();++iter)
{
}
```

反例：

```
auto iter = map1.begin();
for(;iter!=map1.end();iter++)
{
}
```

【强制】函数参数定义采用现代风格

检查方式：工具

原因：提升代码可读性

正例：

```
int main(int argc, char * argv[])
{
    …
}
```

反例：

```
int main(argc, argv)
int argc;
char * argv[];
{
    …
}
```

【推荐】参数的书写要完整，不要贪图省事只写参数的类型而省略参数名字

检查方式：人工检查

原因：提升代码可读性

正例：

```
void SetValue(int width, int height);
```

反例：

```
void SetValue(int, int);
```

【参考】参数命名要恰当，顺序要合理

检查方式：人工检查

原因：提升代码可读性

【推荐】如果参数是指针，且仅作输入参数用，则应在类型前加 const，以防止该指针在函数体内被意外修改

检查方式：人工检查

原因：防止该指针在函数体内被意外修改

【推荐】不要省略返回值的类型，如果函数没有返回值，那么应声明为 void 类型

检查方式：工具

原因：提升代码可读性

【强制】return 语句不可返回指向“栈内存”的“指针”

检查方式：工具

原因：导致外部使用的时候，栈空间已经被释放

反例：

```
char * Func(void)
{
    char str[30];
    …
    return str;
}
```

【推荐】不要将正常值和错误标志混在一起返回。正常值用输出参数获得，而错误标志用返回值返回

检查方式：人工检查

原因：提升代码可读性

【推荐】函数的功能要单一，不要设计多用途的函数

检查方式：人工检查
原因：提升代码可读性

【参考】函数体的规模要小，尽量控制在 200 行代码之内
检查方式：人工检查
原因：提升代码可读性

【推荐】相同的输入应当产生相同的输出。尽量避免函数带有“记忆”功能
检查方式：人工检查
原因：提升代码可读性

【推荐】避免函数有太多的参数，参数个数尽量控制在 5 个以内
检查方式：人工检查
原因：如果参数太多，在使用时容易将参数类型或顺序搞错。并且，参数过多对性能也会有影响

【推荐】对于存取函数以及其他函数体比较短，性能关键的函数鼓励内联；函数体较长是不建议内联（一般函数体不超过 10 行）
检查方式：人工检查
原因：过长的内联函数可能会降低程序性能

【强制】需要确保迭代语句或选择语句在{}内
检查方式：工具
原因：代码可读性降低
反例：

```
while (data_available)        //建议添加括号
process_data();

if (flag_1)
if (flag_2)            //建议添加括号
    action_1();
else
```

```
    action_2();
```

【推荐】无限循环

检查方式：工具

原因：程序不会继续往下执行

反例：

```
while(true)
{
    …
}
```

【推荐】无法访问的代码

检查方式：工具

原因：冗余代码，可能是因为写代码疏忽导致

反例：

```
int16_t UnReach(int16_t para)
{
    …
    return para;
    para++;    //函数已经返回，后面的代码不会被执行都
}

enum ec { RED, BLUE, GREEN} col;
if(col <= GREEN) //总是为真，else 分支会跑不到
{
    // Will always get here
}
else
{
    //Will never get here
}
```

【参考】禁止递归调用

检查方式：工具

原因：代码可读性降低

【强制】不得有无效代码

检查方式：工具

原因：提升代码可读性

反例：

```
extern volatile uint16_t v;
extern char *p;
void f ( void )
{
    uint16_t x;
    (void)v
    ( int32_t )v; //不合规
    v >> 3;       //不合规
    x = 3;     //不合规,x 后面没有访问
    *p++;      //*操作没有使用
}

int16_t DealCode(int16_t para)
{
    int16_t local = 99;
    para = para + local;
    local = para; //无效代码
    if(0 == local)    //无效代码
    {
        local++;   //无效代码
    }
    return para;
}
```

【推荐】使用了浮点数作循环计数器

检查方式：工具

原因：不同系统具有不同的浮点数精度限制，用浮点数作为循环计数器会降低程序的可移植性

【强制】控制表达式不应是不变的

检查方式：工具

原因：表达式没有意义

反例：

```
if (2 > 3) //永远不成立
{
}
uint8_t a;
if (a < 0) //无符号不可能小于 0
{
}
```

7　注释

【推荐】建议采用注释文档提取工具（如 doxygen），并按工具标准对函数、类、变量的声明进行注释

检查方式：人工检查

原因：提升代码可读性，同时地方便后期直接生成开发文档

【推荐】注释应当准确、易懂，防止有二义性。错误的注释不但无益反而有害

检查方式：人工检查

原因：提升代码可读性

【推荐】边写代码边注释，修改代码同时修改相应的注释，以保证注释与代码的一致性。不再有用的注释要及时删除

检查方式：人工检查

原因：提升代码可读性

【推荐】注释是对代码的“提示”，而不是文档。程序中的注释不可喧宾夺主，注释太多了会让人眼花缭乱

检查方式：人工检查

原因：提升代码可读性

【推荐】一目了然的语句不加注释

检查方式：人工检查

原因：提升代码可读性

反例：

```
i++;    /* i 加 1 */
```

多余的注释

【强制】对于全局数据（全局变量、常量定义等）必须要加注释

检查方式：人工检查

原因：提升代码可读性

【推荐】注释采用中文，尽量避免在注释中使用缩写，特别是不常用缩写

检查方式：人工检查

原因：提升代码可读性

【推荐】注释的位置应与被描述的代码相邻，可以与语句在同一行，也可以在上行，但不可放在下方，与代码同一行的注释不宜太长。同一结构中不同域的注释要对齐

检查方式：人工检查

原因：提升代码可读性

【推荐】当代码比较长，特别是有多重嵌套时，应当在一些段落的结束处加注释，便于阅读

检查方式：人工检查

原因：提升代码可读性

正例：

```
if (…)
{
```

```
        …
        while (…)
        {
          …
        } /* while 语句结束 */
       …
    } /* if 语句结束 */
```

【强制】//行末不得添加反斜杠继续注释下行代码

检查方式：工具

原因：容易因误操作屏蔽下行代码

正例：

```
//test Fn2
//input param
int32_t Fn2 (int32_t a)
{
    …
}
```

反例：

```
//test Fn1\
input param
int32_t Fn1 (int32_t a)
{
    …
}
```

【推荐】字符序列/ *和//不得在注释中使用

检查方式：工具

原因：注释代码不建议嵌套

8　类

【推荐】尽量不要使用多重继承。真正需要用到多重继承的情况少之又少．有时多重实现继承看上去是不错的解决方案，但这时你通常也可以找到一个更明确，更清晰的不同解决方案。

检查方式：人工检查

原因：多重集成过于复杂，存在二义性等问题

【推荐】不要定义隐式类型转换．对于单参数构造函数，请使用 explicit 关键字

检查方式：人工检查

原因：隐式类型转换会隐藏类型不匹配的错误．有时，目的类型并不符合用户的期望，甚至用户根本没有意识到发生了类型转换。同时，隐式类型转换会让代码难以阅读。

正例：

```
class A {
public:
    explicit A(int a)
    {
    }
};

A a = ‘s’ ;  // explicit 关键字不允许’s’隐式转换成 int 类型
```

【推荐】不要在构造函数中进行可能会失败的初始化操作，可以增加专门的 Init 方法进行初始化

检查方式：人工检查

原因：构造函数没有返回值，无法检查初始化结果

【强制】不要在构造函数中调用虚函数

检查方式：工具

原因：不要在构造函数和析构函数中调用虚函数，因为这种情况下的虚函数调用不会调用到外层派生类的虚函数；对象的虚函数表地址在对象的构造和析构过程

中会随着部分类的构造和析构而发生变化

【推荐】类成员变量使用初始化列表就地初始化(c++11)

检查方式：人工检查

原因：可以避免遗漏成员变量初始化

正例：

```
class A {
public:
    A()
    {
    }

private:
    int m_ivalue{ 0 };
};
```

【推荐】尽量使用对象类型成员变量，而不是对象指针类型

检查方式：人工检查

原因：可以避免动态构造对象，以及使用时的对象指针有效性检查

反例：

```
class A {
public:
    int Init()
    {
        m_lpB = new B();
        …
        return 0;
    }

private:
    B * m_lpB{ NULL };
};
```

说明：A 中有一个成员变量是 B 的对象指针，并且在初始化中进行构造。这种场景下，建议将该成员变量类型修改为对象实例，而不是对象指针。

【参考】不得重载&&，|| 和，操作符

检查方式：工具

原因：导致使用的时候存在歧义

【参考】不得重载操作符&

检查方式：工具

原因：导致使用的时候存在歧义

反例：

```
calss
{
public:
    A * operator &();
}
```

【强制】派生类的函数或变量隐藏了基类的函数或变量

检查方式：工具

原因：提升代码可读性

正例：

```
class A
{
public:
    virtual ~A () = default;
    virtual void F () {}
};
class B: public A
{
public:
    virtual ~B () = default;
    void F () override {}
```

```
};
```

反例：

```
class A
{
public:
    virtual ~A () = default;
    void F (){}
};
class B: public A
{
public:
    virtual ~B() = default;
    void F(){} //和基类的函数一样
};
```

【参考】虚函数应该使用下面三个关键子中的一个 1.virtual, 2.override, 3.final

检查方式：工具

原因：override 作用：在函数比较多的情况下可以提示读者某个函数重写了基类虚函数（表示这个虚函数是从基类继承，不是派生类自己定义的）；强制编译器检查某个函数是否重写基类虚函数，如果没有则报错。finnal 作用：一个类或结构体不希望被继承；一个虚函数不希望被 override。

反例：

```
class A
{
public:
    virtual ~A(){}
    virtual void F() = 0;
    virtual void G() final = 0;//同时使用 virtual 和 final
};

class B :  public A
```

```
{
public:
    ~B() override {}
    virtual void F () override; //同时使用virtual和override
};
```

【参考】每个覆盖的虚函数必须用 override 或 final 声明符声明

检查方式：工具

原因：override 作用：在函数比较多的情况下可以提示读者某个函数重写了基类虚函数（表示这个虚函数是从基类继承，不是派生类自己定义的）；强制编译器检查某个函数是否重写基类虚函数，如果没有则报错。finnal 作用：一个类或结构体不希望被继承；一个虚函数不希望被 override。

正例：

```
class A
{
public:
    virtual ~A() {}
    virtual void F()= 0;
    virtual void G()  {}
};
class C: public A
{
public:
    ~C() override{}
    void F() override{}
    void G() override{}
};
```

反例：

```
class A
{
public:
    virtual ~A () {}
```

```
    virtual void F () = 0;
    virtual void G (){}
};

class B :  public A
{
public:
    ~B(){}          //不合规
    void F(){}          //不合规
    virtual void G(){}    //不合规
};
```

【参考】用户定义的赋值运算符不得为虚函数

检查方式：工具

原因：如果在基类 A 中将重载的运算符声明为虚拟的，则在其子类 B 和 C 中需要为替代程序提供相同的参数列表。这允许调用类 B 的赋值运算符，该赋值运算符采用 C 类型的参数，这可能导致未定义的行为。

反例：

```
class A
{
public:
    virtual A& operator=(A const& oth) = 0;
    virtual A& operator+=(A const& rhs) = 0;
};
```

【参考】虚函数不能被纯虚函数覆盖

检查方式：工具

原因：用法错误

反例：

```
class B
{
public:
```

```
    virtual void foo ();
};

class C:public B
{
public:
    virtual void foo () = 0;
}
```

9 类型转换

【强制】隐式转换可能会导致数据丢失

检查方式：工具

原因：可能存在溢出

反例：

```
int8_t Fn1(int8_t x, int8_t y)
{
    return x + y; //x + y 可能会导致数据溢出
}
```

【推荐】0 和 NULL 之间隐式转换

检查方式：工具

原因：该写法会导致代码可读性降低

反例：

```
void f1(int32_t)
void f2(int32_t *)
void f3()
{
    f1(NULL);
    f2(0);
}
```

【推荐】表达式发生隐式转换

检查方式：工具

原因：可能导致期望结果不对

正例：

```
void f ()
{
    int32_t s32;
    int8_t s8;
    s32 = static_cast<int32_t>(s8) + s8;
}
```

反例：

```
void f ()
{
    int32_t s32;
    int8_t  s8;
    s32 = s8 + s8;
}
```

【强制】隐式转换会改变符号

检查方式：工具

原因：提升代码可读性

正例：

```
void f()
{
    int8_t s8;
    uint8_t u8;
    u8 = static_cast<uint8_t>(s8) + u8;
}
```

反例：

```
void f ()
{
    int8_t s8;
    uint8_t u8;
```

```
    s8 = u8;
    u8 = s8 + u8;
}
```

【推荐】表达式强制转换为不同符号的基础类型

检查方式：工具

原因：类型不匹配可能导致期望结果不对

正例：

```
void f ()
{
    int8_t s8;
    uint8_t u8;
    s8 = static_cast<int8_t>(u8 ) + static_cast<int8_t> (u8);
}
```

反例：

```
void f ()
{
    int8_t s8;
    uint8_t u8;
    s8 = static_cast<int8_t>(u8 + u8);
}
```

【强制】指向函数的指针被强制转换为其它类型

检查方式：工具

原因：导致函数指针使用出现问题

反例：

```
void f(int32_t)
{
    reinterpret_cast< void (*)() > (&f);
    reinterpret_cast< void * > (&f);
}
```

【参考】整型或者 void*类型强制转换成对象指针

检查方式：工具

原因：避免类型错误导致程序异常

反例：

```
struct S
{
    int32_t i;
    int32_t j;
};
void f (void * v, int32_t i)
{
    S *s1 = reinterpret_cast<S *>(v);
    S *s2 = reinterpret_cast<S *>(i);
}
```

【推荐】函数指针不得和其它类型进行转换

检查方式：工具

原因：避免类型错误导致程序异常

反例：

```
typedef void (*fp16) (int16_t n);
typedef void (*fp32) (int32_t n);
fp16 fp1 = NULL;
fp32 fp2 = (fp32) fp1;
fp16 fp3 = ( fp16 ) 0x8000;
```

【推荐】指针类型不得在不同的对象之间进行转换

检查方式：工具

原因：导致函数指针使用出现问题

【推荐】不能在数字类型和 void 类型的指针之间相互转换

检查方式：工具

原因：将整数转换为指向 void 的指针可能会导致指针未正确对齐，导致不确定的行为

反例：

```
void * p;
uint32_t u;
p = (void *)0x1234u;
p = (void *)1024.0f;
u = (uint32_t)p;
```

【推荐】在对象的指针和非整数算术类型之间不得执行强制转换

检查方式：工具

原因：将基本布尔值、基本字符或基本枚举类型转换为指向的指针对象可能会导致指针未正确对齐，从而导致未定义的行为

反例：

```
int16_t * p;
float f;
f = (float) p;
p = (int16_t *) f;
```

【推荐】不要将指针转换为更严格对齐的指针类型

检查方式：工具

原因：对象对齐方式可能会被改变

正例：

```
void func (void)
{
    char c = 'x';
    int32_t i = c;
    int32_t *ip = &i;
}
```

反例：

```
void func (void)
{
```

```
    char c = 'x';
    int32_t *ip = (int32_t *)&c;
    char *cp = (char *)ip;
}
```

【参考】确保浮点转换在新类型的范围内

检查方式：工具

原因：防止转换后值不正确

反例：

```
void func (float f)
{
    int32_t a;
    a = f;
}
```

【参考】将整数值转换为浮点型时丢失精度

检查方式：工具

原因：防止转换后值被不正确

反例：

```
int32_t main (void)
{
    int64_t big = 1234567890;
    float approx = bix;  //float 表示不了 10 位精度
    return 0;
}
```

10 指针和数组

【强制】禁止把指针当数组使用

检查方式：工具

原因：需要结合数组大小一起使用，避免访问越界

正例：

```
void F1 (uint8_t *p1, uint8_t p2[])
```

```
{
    uint8_t *p4;
    p2[5] = 0;
    p4 = &p2[6];
}
```

反例：

```
void F1 (uint8_t *p1, uint8_t p2[])
{
    uint8_t *p3;
    p1 = p1 + 5;
    p1[5] = 0;
    p3 = &p1[5];
}
```

【推荐】指针访问越界

检查方式：工具

原因：会导致程序异常

反例：

```
void f1 ()
{
    int32_t a1[10];
    int32_t *p3 = &a1[12];
}
```

【强制】对类型不同的指针进行算术运行可能产生溢出

检查方式：工具

原因：结果可能溢出

正例：

```
void f1 (int32_t *ptr)
{
    int32_t a1[10];
    int32_t a2[10];
```

```
    int32_t *p1 = &a1[1];
    int32_t *p2 = &a2[10];
    ptrdiff_t diff;
    diff = p1 - a1;
    diff = p2 - a2;
}
```

反例：

```
void f1 (int32_t *ptr)
{
    int32_t a1[10];
    int32_t a2[10];
    int32_t *p1 = &a1[1];
    int32_t *p2 = &a2[10];
    ptrdiff_t diff;
    diff = p1 - p2;
    diff = ptr - p1;
}
```

【强制】使用零长度数组

检查方式：工具

原因：容易引入问题

反例：

```
struct s
{
    uint16_t len;
    uint32_t data[];
};
```

【推荐】使用可变长度数组类型

检查方式：工具

原因：函数栈大小未确定

反例：

```
void f (int16_t n)
{
    uint16_t vla[n];
}
```

【推荐】使用指定下标初始化数组的时候，数组的大小应明确指定

检查方式：工具

原因：提高代码可读性

反例：

```
int32_t a1[ ] = { [ 0 ] = 1 };
```

11 异常

【参考】仅应抛出从 std :: exception 派生的类型的实例

检查方式：工具

原因：提高代码可读性

反例：

```
void F1()
{
    throw -1;
}
void F2()
{
    throw nullptr;
}
```

【强制】抛出异常对象不能是指针

检查方式：工具

原因：如果抛出了指针类型的异常对象，并且该指针指向动态创建的对象，则可能不清楚哪个函数负责销毁该对象以及何时销毁该对象，这可能会导致内存泄漏；

如果抛出了指针类型的异常对象，并且该指针引用了一个自动变量，则它允许在销毁变量后使用变量，从而导致不确定的行为。

正例：

```
void Fn(int16_t i)
{
    A a1;
    A &a2 = a1;
    A *a3 = new A;

    if(i < 10)
    {
        throw a1;
    }
    else if(i < 20)
    {
        throw a2;
    }
}
```

反例：

```
class A
{
    …
};
void Fn(int16_t i)
{
    A a1;
    A &a2 = a1;
    A *a3 = new A;

    if (i < 10)
    {
        throw &a1;//不合格
    }
    else if (i < 20)
```

```
        {
            throw a3;  //不合格
        }
    }
```

【强制】不得显式抛出 NULL 异常

检查方式：工具

正例：永远不会抛出 null 指针常量，因此只会被整数处理程序捕获

```
    try
    {
        throw (static_cast<const cahr_t *>(NULL));
    }
```

反例:

```
    try
    {
        throw (NULL);
    }
```

【强制】不得使用动态异常规范

检查方式：工具

原因：此功能在 2011 C ++语言标准中已弃用

正例：

```
    void F1() noexcept;
    void F3() noexcept(false);
```

反例:

```
    void F2() throw();
    void F4() throw(std::runtime_error);
```

七、 其它规范

【强制】防止引用已经释放的内存空间

检查方式：工具

原因：会导致程序异常

反例：

```
char *p = (char *) malloc(100);
strcpy(p, “hello”);
free(p);        /* p 所指的内存被释放，但是 p 所指的地址仍然不变 */
…
if (p != NULL)    /* 没有起到防错作用 */
{
   strcpy(p, “world”); /* 出错 */
}
```

【推荐】对于运行期会被频繁调用的代码中动态申请的内存，应进行内存泄露检查

检查方式：工具

原因：频繁申请内存的地方，最容易发生内存泄露

反例：

1)同一函数内进行内存申请与释放时，其间的有多个 retrun 点，有的 return 点忘了进行内存释放，导致内存泄露。

2）出现野指针，如:

```
char *p = (char *) malloc(100);
…
p = other();  /*原来所指内存再也没有指针引用*/
…
free(p);
```

3）重分配内存失败，如:

```
char *p = (char *) malloc(100);
…
p = (char *) realloc(p, iNewSize);  /*realloc 失败时，p 被置为 NULL，原有内存丢失*/
if (p) …
free(p);
```

【强制】malloc、free 与 new、delete 必须配对使用，即 malloc 分配的内存不

能用 delete 释放，new 的内存不能用 free 释放。并且，new[]需要用 delete[]释放。

检查方式：工具

原因：比如 new[]申请的内存，如果用 delete 释放，可能会导致内存泄露

【强制】进行内存、字符串拷贝操作（如 strcpy、sprintf、memcpy）时，检查并防止内存越界，同时需要注意源和目的之间内部不能重叠。

检查方式：工具

原因：会导致程序异常

【强制】跨平台 C++编程时，字符串操作函数（如 strlen,strcpy）要求所操作的字符串不能为 NULL(若为 NULL，在非 windows 平台下会直接异常退出)，即函数原型中参数类型为 const char *，则默认要求调用者提供非 NULL 的实参；此规则也适用所有用 const 修饰的指针。

检查方式：工具

原因：会导致程序异常

【强制】函数中申请的（为打开文件而使用的）文件句柄，在函数退出前要关闭。

检查方式：工具

原因：会导致句柄泄露，进而引起程序异常

【推荐】打开编译器的所有告警开关后对程序进行编译。

检查方式：人工检查

原因：可以在编译期发现一些程序异常

【推荐】结构体变量直接赋值，左值和右值类型相同，两个定义（部分）相同的结构体也不能直接赋值

检查方式：人工检查

原因：如果结构体定义发生变化后两个结构体定义不再相同，那么赋值可能无法得到预期效果，同时编译器无法帮助检查这类异常

反例：

```
struct A {
    int iA1;
    double dA2;
};

struct B {
    int iB1;
    double dB2;
    char sB3[32];
};

A * lpA;
B * lpB;
```

希望将 lpB->iB1 赋值给 lpA->iA1，lpB->dB2 赋值给 lpA->dA2。

```
*lpA = *(A *)lpB
```

该用法一般没有问题，但是如果 B 的定义发生改变，比如修改为：

```
struct B {
    int iB1;
    char sB3[32];
    double dB2;
};
```

那么该语句就不会得到我们期望的效果，并且编译器不会告警或报错。

【推荐】禁止定义对象类型的全局变量，因为多编译单元中的全局变量执行时的构造和析构顺序是未明确的

检查方式：人工检查

原因：可以在编译期发现一些程序异常

【推荐】内存分配与释放，互斥锁的加锁与释放尽量保同一层次的代码内完成，以便检查是否成对。

检查方式：工具

原因：便于检查是否成对，从而保证不会出现内存泄露以及锁资源未释放的问题

【推荐】尽可能用 sizeof(varname) 代替 sizeof(type)

检查方式：人工检查

原因：类型变化时，程序行为仍然符合预期

正例：

```
struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
```

反例：

```
struct sockaddr_in addr;
memset(&addr, 0, sizeof(struct sockaddr_in));
```

【推荐】类型应明确规定

检查方式：工具

原因：提升代码可读性

反例：

```
extern x;          //没有定义类型
const y;           //没有定义类型
extern f (void);      //返回类型未明确
extern void g (char c, const k);    //输入参数 k 的类型未明确
```

【推荐】在枚举数列表中，隐式指定的枚举常数值应唯一

检查方式：工具

原因：不符合编码习惯，容易出错

正例：

```
enum colour { red = 3, blue, green = 5, yellow = 5 };
```

反例：

```
enum colour { red = 3, blue, green, yellow = 5 };    //green    和
yellow 都是 5
```

【推荐】集合或联合的初始化应放在大括号中

检查方式：工具

原因：提升代码可读性，同时避免出错

正例：

```
int16_t y[ 3 ][ 2 ] = { { 1, 2 }, { 0 }, { 5, 6 } };
```

反例：

```
int16_t y[ 3 ][ 2 ] = { 1, 2, 0, 0, 5, 6 };
```

【推荐】对象避免重复初始化

检查方式：工具

原因：不利于的代码维护

反例：

```
int16_t a3[ 5 ] = { [ 0 ] = -5, [ 1 ] = -4, [ 2 ] = -3, [ 2 ] = -2,
[ 4 ] = -1 };
```

【推荐】表达式的值及其持续的副作用应相同

检查方式：工具

原因：表达式产生副作用不一样，容易出错

正例：

```
uint16_t f (uint16_t y)
{
    uint16_t temp = y;
    temp = y + 0x8080U;
    return temp;
}

void g (void)
{
    bool b;
    if (b && (a == f (x)))   //f 函数有没有被调用到都不会产生持续影
响
    {
    }
```

```
}
```

反例：

```
uint16_t f(uint16_t y)
{
    static uint16_t temp = y;
    temp = y + 0x8080U;
    return temp;
}

void g (void)
{
    bool b;
    if (b && (a == f (x)))   //f(x)可能被调用，也可能不被调用，并且f 函数里面存在静态变量
    {
    }
}
```

【强制】不得尝试写入以只读方式打开的流

检查方式：工具

原因：文件模式不匹配，程序异常

反例：

```
void fn (void)
{
    FILE *fp = fopen ("tmp", "r");
    ( void )fprintf(fp, "What happens now?");
    ( void )fclose (fp);
}
```

【强制】文件关闭之后不得再使用

检查方式：工具

原因：句柄无效，程序异常

反例：

```
void fn (void)
{
    FILE *fp;
    void *p;
    fp = fopen ("tmp", "w");
    if (fp == NULL)
    {
        error_action ();
    }
    fclose (fp);
    fprintf (fp, "?");
    p = fp;
}
```

【推荐】不要为同一函数或对象创建不兼容的声明

检查方式：工具

原因：申明不一致可能导致符号名不同，无法链接

反例：

```
/* In a.c */
extern int32_t i;
int32_t f(void)
{
    return ++i;
}

/* In b.c */
uint8_t i;    //相同外部变量类型必须一致
```

【推荐】不得比较填充数据

检查方式：工具

原因：由于结构体对其引入的内存填充部分，其数据值是不确定的，直接进行比

较可能会无法得到预期效果（有效部分是相等的，但是由于填充部分不相等，导致最终结果为不相等）。正确使用方式是直接比较结构体字段，或者在结构体初始化以后，再进行内存比较

正例：

```
#include <string.h>
struct s
{
    char c;
    int i;
    char buffer[13];
};

void compare(const struct s *left, const struct s *right)
{
    if((left && right) &&
    (left->c == right->c) &&
    (left->i == right->i) &&
    (0 == memcmp(left->buffer, right->buffer, 13)))
    {
        …
    }
}
```

反例：

```
#include <string.h>
struct s
{
    char c;
    int i;
    char buffer[13];
};

void compare(const struct s *left, const struct s *right)
```

```
{
    if (0 == memcmp(left, right, sizeof(struct s)))
    {
        …
    }
}
```

【推荐】不能复制文件对象

检查方式：工具

原因：文件对象指针由 fopen 返回，通过 fread、fwrite 等方法（文件对象指针作为入参）来完成文件读写操作。应用完全没必要复制文件对象，并且 fwrite、fclose 等方法传入拷贝的文件对象会引起程序异常终止。

正例：

```
#include <stdio.h>
int32_t main(void)
{
  FILE* ofile = fopen("a.txt", "wb+");
  if (fputs("Hello, World!\n", ofile) == EOF)
  {
    /* Handle error */
  }
  fclose(ofile);
  return 0;
}
```

反例：

```
#include <stdio.h>
int32_t Func(FILE copy_file)
{
  fwrite("Hello, World!\n", 1, 12, &copy_file);      // 程序异常终止
  return 0;
}
```

```
int32_t main(void)
{
  FILE* ofile = fopen("a.txt", "wb+");
  if (fputs("Hello, World!\n", ofile) == EOF)
  {
      /* Handle error */
  }
  Func(*ofile);
  return 0;
}
```

【推荐】不要在选择语句中执行赋值语句

检查方式：工具

原因：提升代码可读性，同时避免出错

正例：

```
if (a == b)
{
    …
}
```

反例：

```
if (a = b)
{
    …
}
```

【强制】不建议使用 system()

检查方式：工具

原因：使用 system()函数可能会导致可攻击的漏洞，最坏的情况是允许执行任意系统命令。调用 system()具有高风险的情况包括：

1）传递源于受污染源的未初始化或未正确清理的命令字符串

2）如果指定的命令没有路径名，并且攻击者可以访问命令处理器路径名解析机

制

3) 如果指定了可执行文件的相对路径，并且攻击者可以访问当前工作目录

4) 如果指定的可执行程序被攻击者欺骗(或替换)

【推荐】不要使用 rand()获取随机数

检查方式：工具

原因：C 标准 rand()函数不能保证所生成随机序列的质量，由 rand()的某些实现生成的数字的周期相对较短，并且这些数字是可以预测的

【推荐】unsigned char 和 signed char 建议使用数字进行赋值

检查方式：工具

原因：提升代码可读性

正例：

```
int8_t c = 10;
uint8_t d = 12U;
```

反例：

```
int8_t a = 'a';
uint8_t b = '\n';
```

【推荐】sizeof 用在有副作用的表达式中

检查方式：工具

原因：表达式不会被执行，使程序可读性降低

反例：

```
int32_t i;
size_t  k;
k = siezeof(i == 1234);
k = sizeof(i++);
```

【强制】避免除 0 操作

检查方式：工具

原因：程序报除 0 错误，程序终止

反例：

```
int32_t g(int32_t x)
{
    x = x - 5;
    return 100 / x;        //x 为 0
}

void f()
{
    int32_t a = 3;
    a += 2;
    a = g(a);
}
```

【推荐】变量定义了未使用

检查方式：工具

原因：容易出错，不利于代码后期维护

反例：

```
extern void TestFn(int16_t a, int16_t b);
void WithUnusedVal(int16_t a)
{
    int16_t UnusedVar;        //变量未使用
    TestFn(a, 0);
}
```

【推荐】输入参数未使用

检查方式：工具

原因：容易出错，不利于代码维护

反例：

```
int16_t f1(int16_t a, int16_t b)
{
    return a;
}
```

【强制】不建议使用 long double 类型

检查方式：工具

原因：不同编译器中 long double 类型的大小存在差异，不建议使用，避免引入兼容性问题

【参考】谨慎使用八进制和十六进制

检查方式：工具

原因：提升代码可读性

反例：

```
int32_t a[8];
a[0] = 053;
a[1] = 071;
```

【推荐】不要在代码块中声明函数

检查方式：工具

原因：提升代码可读性

反例：

```
void b1()
{
    void f1 ();
}
```

【强制】变量、函数输入参数、返回值的声明和定义必须一致

检查方式：工具

原因：申明不一致可能导致符号名不同，无法链接

反例：

```
typedef int32_t INT;
INT i;
extern uint32_t i;

extern void f(signed int)
```

```
void f(int);
```

【参考】对于不变的变量添加 const 限制

检查方式：工具

原因：方便编译器优化，提升运行效率；编译器能够检查非法赋值，避免程序异常

【强制】有返回值的函数忘记返回值

检查方式：工具

原因：导致程序返回值不确定

正例：

```
int32_t F2()
{
    …
    return n;
}
```

反例：

```
int32_t F1()  //函数忘记返回值
{
}
```

【强制】避免运行结果依赖运行顺序的代码

检查方式：工具

原因：提升代码可读性

正例：

```
void F2(uint8_t (&arr)[10], uint8_t idx)
{
    uint16_t x = arr[idx] + idx;
    idx++;
}
```

反例：

```
void F1(uint8_t (&arr)[10], uint8_t idx)
```

```
{
    uint16_t x = arr[idx] + idx++;
}
```

八、 64 位程序移植规范

【推荐】结构体定义最好是 8 字节对齐，不够的用占位符补全，防止以后移植到 64 位系统出问题

检查方式：人工检查

原因：避免由于不同平台自动对齐方式的差异导致数据错位

【推荐】如果依赖第三方库，那么需要确认其是否支持 64 位移植或有现成的 64 位库

检查方式：人工检查

原因：避免应用升级到 64 位时，无法正常使用

【强制】涉及到磁盘或网络读写结构体时，结构体定义中切勿使用 long、size_t、指针等类型，因为这些类型在 32 位和 64 位环境中的长度是不一样的，会产生不兼容的问题，推荐使用标准长度的整数

检查方式：人工检查

原因：不同环境下，long、size_t 类型长度不一致，容易产生兼容性问题

【强制】在代码执行逻辑的内部，切勿假设指针是 4 个字节，而应该用 sizeof（比如 memset 一个指针数组）

检查方式：人工检查

原因：64 位下指针是 8 字节

【强制】不要把一个指针赋值给一个整数变量（比如 int），因为可能溢出。

检查方式：工具

原因：64 位下指针是 8 字节，会溢出

九、 建议反馈途径

如对文档有建议，请将修改建议发送至公司 EPG 邮箱 epg@hundsun.com。

十、 修订记录

批准日期	修订说明	修订部门
2007/09/12	初稿通过评审	EPG 研发组
2008/1/22	修改 logo	EPG 研发组
2015/1/14	修改[建议 1-7-1]目录结构，新增 3.1 中的变量命名规则，修改[建议 5-1-2]移植性建议章节	EPG 研发组
2020/10/16	修改[规则 1-1-2]说明 [规则 1-3-1]中增加调用说明 [规则 1-4-1]中增加调用#define 保护 新增[规则 1-4-2] 新增[建议 1-5-1][建议 1-5-2][建议 1-5-3] 新增[建议 1-6-1] [建议 1-7-1] 推荐目录结构调整 调整[规则 3-2-6]描述 新增[规则 3-2-12] 删除[建议 4-5-1]，后续建议编号顺序调整 [规则 4-5-6]描述调整 新增[规则 4-5-8] 调整[规则 4-6-2]描述 调整[规则 4-6-3]描述 [规则 4-6-6]调整为[建议 4-6-1]，后续规则和建议编号顺序调整 [建议 4-6-4]（修改后是 4-6-5） 补充说明 新增[建议 4-6-8] 删除[规则 4-7-6]，后续规则顺序调整 新增 4.8 章节 [规则 5-1-3][规则 5-1-4]补充说明 新增[规则 5-1-8][规则 5-1-9]	EPG 研发组

	新增[建议 5-1-3] [建议 5-1-4] [规则 6-8-3]描述调整	
2021/12/14	3.3.4 头文件结构下增加规范【推荐】对外头文件名称应包含产品简称 修订文档格式	EPG 研发组
2021/12/24	经资管产品架构师提出，技术专家评审确定，3.3.4 头文件结构下增加推荐级别规范。 【推荐】对外头文件名称应包含产品简称 检查方式：人工检查 原因：避免不同产品的头文件冲突，实际使用时出现引用错误 正例：ldp_host.h 反例：host.h	EPG 研发组